SEOAgent.ai — System Architecture & Design Explanation

Autonomous SEO Audit Engine, Technical Scoring, and AI Strategy Agent

Project Name: SEOAgent.ai

Version: 1.0 Production

Repository: github.com/Praveen137-cloud/Seo-agent

Frontend: seo-agent-inky-seven.vercel.app

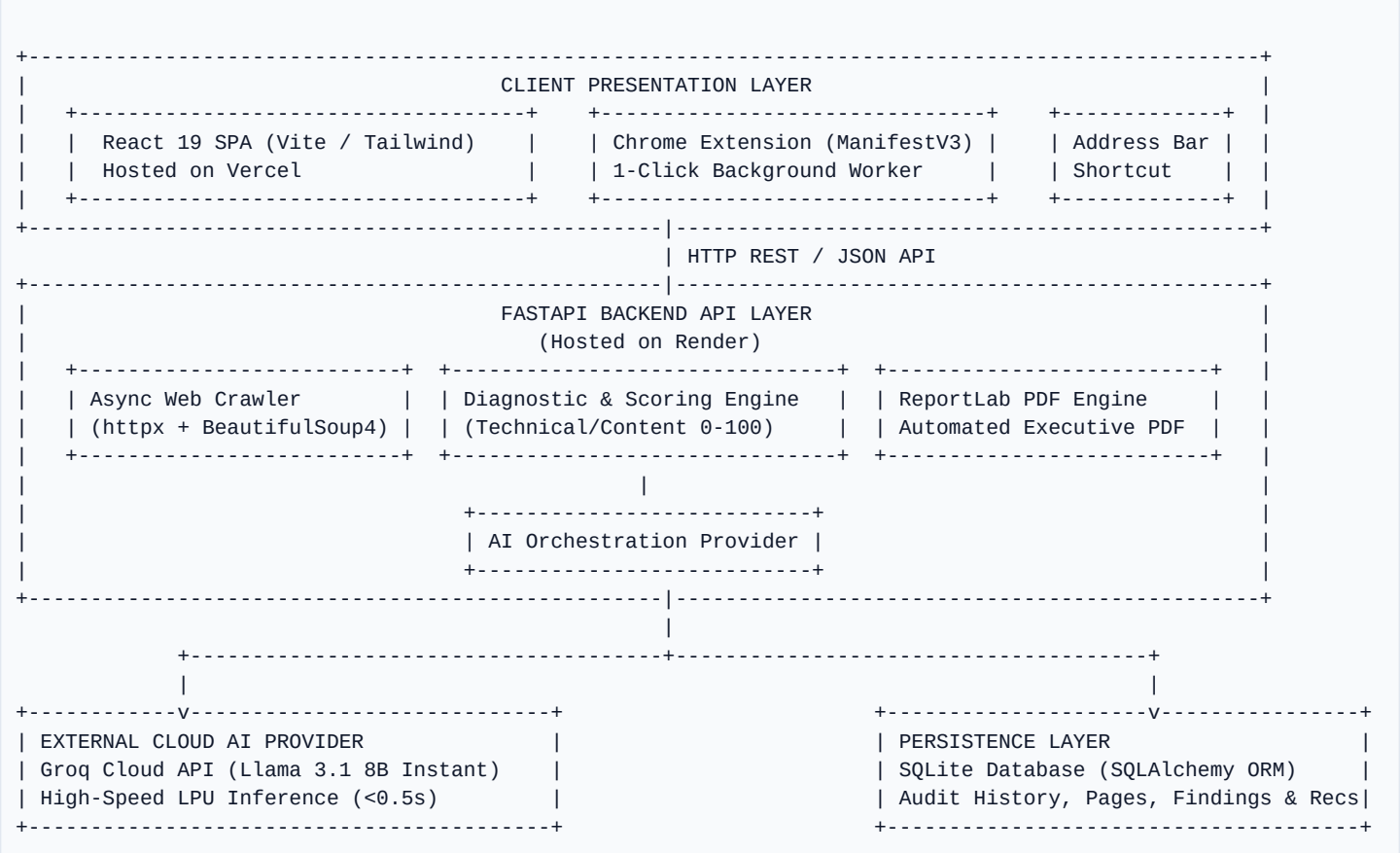
Backend API: seo-agent-api-l0si.onrender.com

AI Provider: Groq Cloud (Llama 3.1 8B)

1. Executive System Overview

SEOAgent.ai is an end-to-end full-stack AI web auditing solution. It combines deterministic multi-page web crawling, algorithmic technical scoring (0-100), and LLM-powered strategic recommendation generation. The architecture strictly separates deterministic data gathering from generative AI reasoning, ensuring high speed, token efficiency, and sub-second execution.

2. High-Level Architecture Diagram



3. Core Component Breakdown

Component	Tech Stack	Role & Responsibilities

Client Frontend	React 19, Vite, Tailwind CSS	Renders live progress bar, score metrics, issue filter tabs (Critical Risk, High Risk, Medium Risk, Low Risk), AI recommendations, and PDF download.
Chrome Extension	Manifest V3, JS Worker	Captures active tab URL on user click and launches instant audit on live Vercel web app.
FastAPI Core	Python 3.12, Uvicorn, CORS	Asynchronous backend router executing background audit tasks, serving status REST APIs, and hosting PDF report generator.
Web Crawler	httpx, BeautifulSoup4	Crawls domain pages asynchronously, extracts titles, meta descriptions, headings, images, internal links, TTFB latency, sitemaps, and robots.txt.
Scoring Engine	Python Rule Engine	Calculates weighted 0-100 health score across 4 pillars: Technical (30%), On-Page (30%), Security (20%), Performance (20%).
AI Provider	Groq Cloud API, Ollama	Constructs context prompts, sends findings to Groq (Llama 3.1 8B), parses structured JSON, and generates ready-to-copy meta content.

4. End-to-End Execution & Data Flow

1. **Request Initiation:** User submits target URL (or 1-clicks Chrome Extension). React frontend calls POST /api/audit.
2. **Async Crawl:** FastAPI creates an Audit ID in SQLite database and launches asynchronous multi-page crawling.
3. **Diagnostic & Scoring:** Page metrics are analyzed against technical rules to compute section scores and categorize issue severity.
4. **Prompt Compression:** Audit findings are compressed into a lightweight JSON schema prompt (reducing token usage by 90%).
5. **Groq LLM Inference:** Groq Cloud API processes the prompt using Llama 3.1 8B in <0.5s and returns structured recommendation JSON.
6. **Persistence & UI Delivery:** Findings, scores, and AI recommendations are saved in SQLite and returned via REST API to React UI and PDF generator.

5. Four-Phase Responsibility Mapping

Phase	Primary Owner	Deliverable Scope
Phase 1	Member 1 — Backend/Crawler	URL validation, BeautifulSoup4 crawler, internal link queue, HTML tag extraction.
Phase 2	Member 2 — Technical SEO & Scoring	HTTPS verification, robots.txt, sitemap.xml, canonicals, 0-100 scoring algorithm, severity & risk tags.
Phase 3	Member 3 — AI Agent & Recommendations	Prompt engineering, Groq Llama 3 integration, plain-English explanations, AI content generation.
Phase 4	Member 4 — Dashboard & PDF	React SPA, Vite setup, Tailwind CSS, Chrome extension, ReportLab PDF generation, history UI.

6. Security & Multi-LLM Resiliency Architecture

To guarantee 100% production availability and avoid secret leakages:
• **Push Protection & Secret Security:** API keys are loaded strictly from environment variables (GROQ_API_KEY) and never committed to source control.
• **Multi-LLM Fallback Chain:** Groq Cloud API (Llama 3.1) $\longrightarrow$ Google Gemini API $\longrightarrow$ OpenAI API $\longrightarrow$ Local Ollama $\longrightarrow$ Smart Rule-based Fallback Generator.
• **Memory Footprint:** Needs 0MB model VRAM on Render free tier (512MB RAM), utilizing Groq's external cloud LPUs for sub-second execution.